YumCart

Order Lifecycle Engine — Queue + Finite State Machine

Jenifer Vincy A
711524BEE027

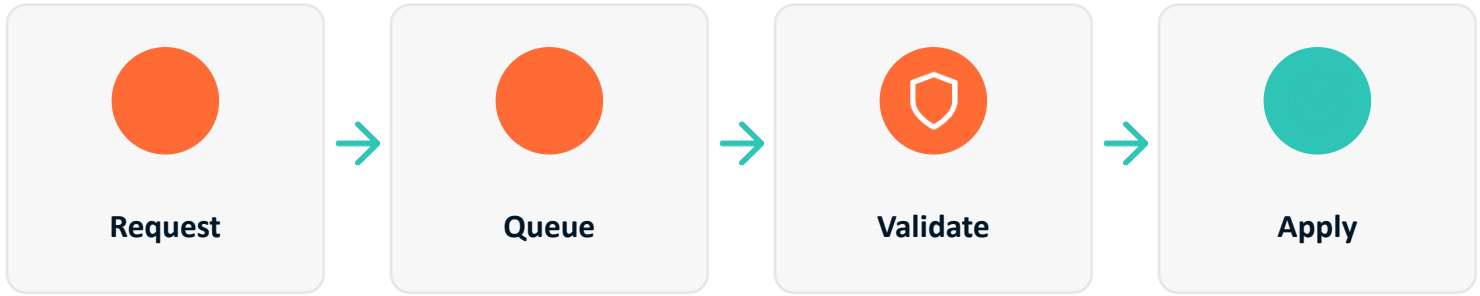


github.com/jenifervincya/YumCart

THE PROBLEM

Every status change is a request, not a command

An order moves through a fixed set of stages, with side branches for cancellation and returns. Instead of updating status directly, every request is queued, validated against a rulebook, then applied or rejected — without ever stopping the system.



Invalid transitions are logged and skipped — one bad event never halts the pipeline.

ORDER STATES

- PLACED
- CONFIRMED
- SHIPPED
- OUT_FOR_DELIVERY
- NOT_DELIVERED
- DELIVERED
- CANCELLED
- RETURNED

● terminal state

DESIGN RATIONALE

Why Queue + State Machine?

Queue

Decouples “a change was requested” from “the change was processed.” Guarantees requests are handled in order, one at a time.

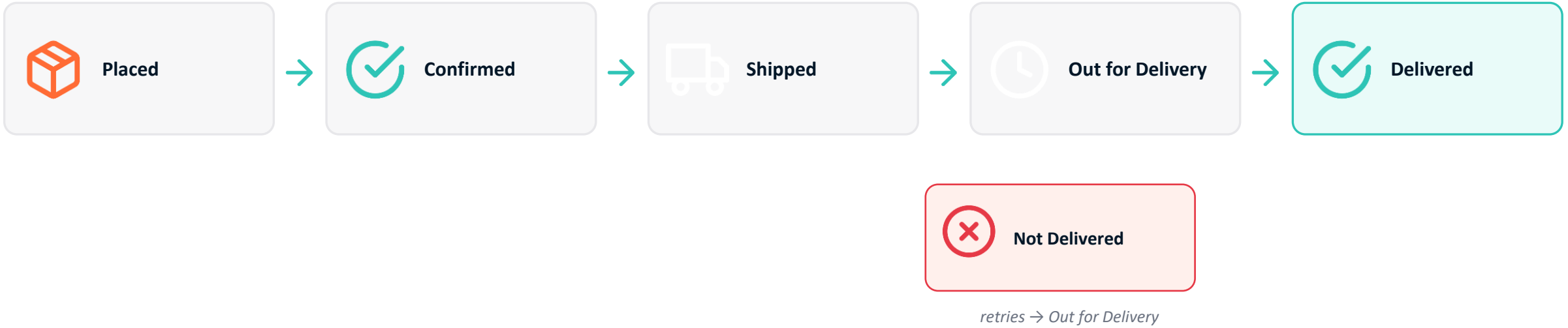
State Machine

Enforces that an order only moves to specific valid next-states from its current one. Arbitrary jumps, like Delivered → Placed, are rejected outright.

Together: a resilient pipeline where every state change is ordered, auditable, and provably valid.

TRANSITION RULES

The order lifecycle, stage by stage



Cancelled

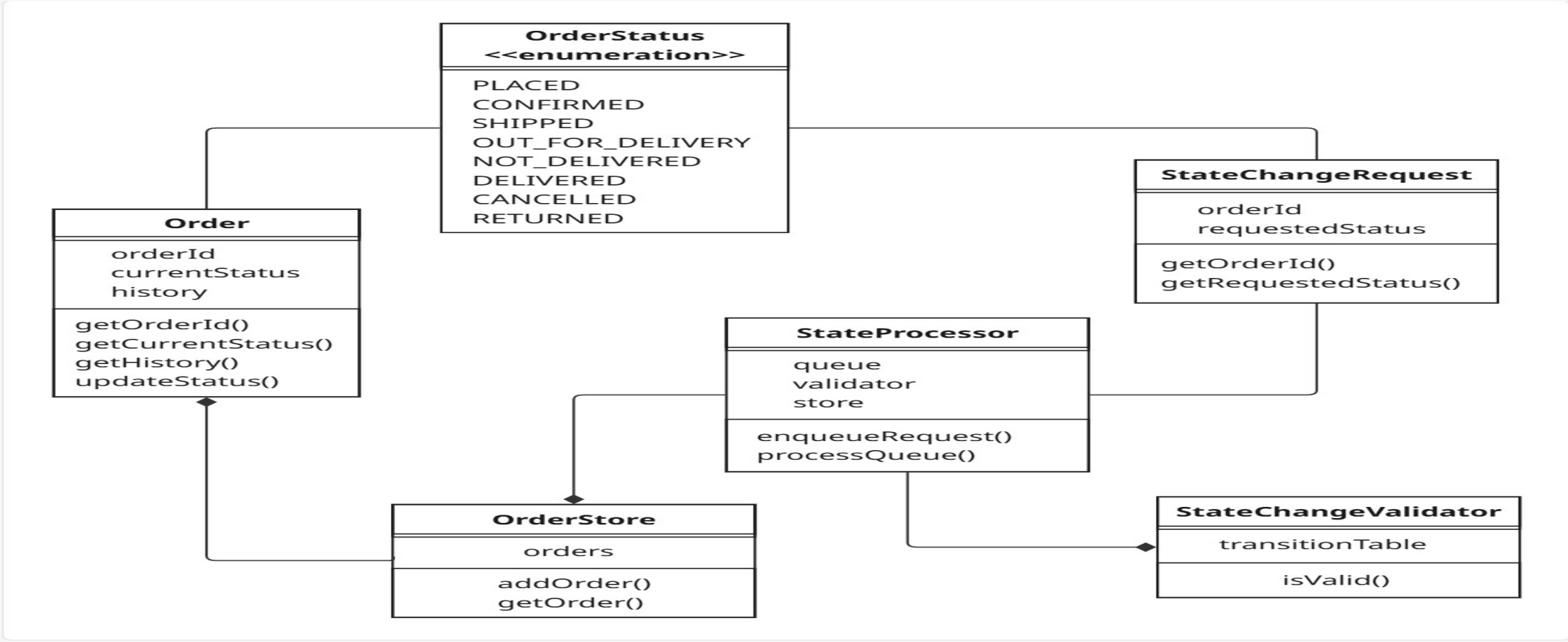
from Placed / Confirmed / Shipped / Not Delivered • terminal, no further transitions

Returned

from Delivered • terminal, no further transitions

ARCHITECTURE

Core order module — class diagram



ARCHITECTURE

Who does what

`Order`

Holds an order's ID, current status, and status history

`OrderStatus`

The fixed enum of valid order states

`StateChangeRequest`

A queued request: which order, which new status

`StateChangeValidator`

Holds the transition rulebook; checks if a move is legal

`OrderStore`

Stores and retrieves Order objects by ID

`StateProcessor`

Drains the queue, validates, and applies or rejects each request

ARCHITECTURE

How the pieces connect

FROM	TO	TYPE	RELATIONSHIP
`StateProcessor`	`StateChangeValidator`	Composition	validates via
`StateProcessor`	`OrderStore`	Composition	stores via
`StateProcessor`	`StateChangeRequest`	Association	consumes
`OrderStore`	`Order`	Composition	holds
`Order`	`OrderStatus`	Association	has current status
`StateChangeRequest`	`OrderStatus`	Association	requests

ROADMAP

Modules being layered on top

Each new module follows the same queue / validator / store pattern where it fits.



User

Customer, RestaurantOwner, DeliveryPartner



Restaurant & Menu

Profiles, menu items, availability



Cart

Single-restaurant cart, quantities, subtotal



Delivery Assignment

Queue-based nearest-partner matching



Payment

Its own FSM: Pending → Success/Failed → Refunded



Notification

Observer pattern hooked into status changes



Pricing

Strategy pattern for discounts, tax, delivery fee

FRONTEND

Two roles, two very different views

Standalone HTML/CSS/JS demo, deployed live via GitHub Pages, sharing mock order data.



Seller / Admin

- Search any order
- Push it to its next status
- Watch the manifest log update live as the queue processes it



Customer

- Read-only view
- Search an order and see current status
- Delivery timeline visible — no controls to change status

Intentionally separated: a customer should never have access to status-change controls.

DESIGN NOTES

Principles behind the choices



Fault-tolerant queue

Rejected transitions don't stop the queue — the system logs and continues, since one bad event shouldn't halt the whole pipeline.



Full audit trail

History is tracked per order so the complete lifecycle can be reviewed end to end.



Single responsibility

OrderStore and StateProcessor are kept separate — storage and queue-processing concerns evolve independently.



Role separation

Seller and customer roles are intentionally split in the frontend for safety and clarity.

See it in action

Try the seller and customer views yourself



Live Demo

jenifervincya.github.io/YumCart



Source Code

github.com/jenifervincya/YumCart

Next up: Delivery Assignment, Payment FSM, and Notification observer wiring.